

Resumen ejecutivo

Este documento detalla las 20 mejoras de seguridad identificadas tras una auditoría exhaustiva del sistema Hospedá. Las mejoras están organizadas por prioridad: 5 críticas (mitigan vectores de ataque reales), 7 importantes (defensa en profundidad) y 8 de hardening (resiliencia y monitoreo).

El sistema ya cuenta con varias protecciones implementadas: validación de monto en webhooks de Mercado Pago, rate limiting en endpoints sensibles, CSP headers, validación de tenant activo, y auditoría de cambios del super-admin. Las mejoras propuestas complementan estas defensas existentes.

Recomendación: Implementar primero las 5 mejoras críticas (Fase 1), que mitigan los riesgos más altos con menor esfuerzo. Las fases 2 y 3 pueden ejecutarse en paralelo sin depender de la 1.

Tabla de mejoras de Seguridad

Listado completo con prioridad, esfuerzo estimado e impacto.
Auditoría completa del sistema Hospedá

#	Mejora	Prioridad	Esfuerzo	Impacto
1	Rate limiting distribuido (Redis)	Crítica	Medio	Previene fuerza bruta
2	Configurar Resend (email service)	Crítica	Bajo	register/forgot-password funcionan
3	Rate limit global en middleware	Crítica	Medio	Previene DoS
4	Cifrar credenciales MP en BD	Crítica	Medio	Protege MP si roban BD
5	Validar IP en webhook MP	Crítica	Bajo	Capa extra en webhook
6	Sentry (monitoreo de errores)	Importante	Bajo	Detección de ataques
7	Error messages genéricos	Importante	Bajo	No filtra info interna
8	Quitar stack trace log	Importante	Trivial	No loggea tokens
9	Eliminar auto-migrate.ts	Importante	Trivial	Quita SQL raw peligroso
10	Zod para validación de inputs	Importante	Alto	Type-safe inputs
11	CSRF tokens en mutations	Importante	Medio	Previene CSRF
12	Google OAuth config en prod	Importante	Bajo	Login Google funciona
13	Backup automático de BD	Hardening	Medio	Recuperación ante desastre
14	Health check endpoint	Hardening	Bajo	Uptime monitoring
15	IP + User Agent en auditoría	Hardening	Bajo	Forenses post-ataque
16	Dependabot (deps auto-update)	Hardening	Trivial	Parches automáticos

#	Mejora	Prioridad	Esfuerzo	Impacto
17	CSP report-uri	Hardening	Bajo	Detección de XSS
18	Rotación de NEXTAUTH_SECRET	Hardening	Trivial	Mitiga secret filtrado
19	2FA para super-admin	Hardening	Medio	Protege panel admin
20	Headers adicionales (COOP, COEP)	Hardening	Trivial	Defense in depth

Fase 1 — Mejoras críticas

Estas 5 mejoras mitigan los vectores de ataque más probables y de mayor impacto. Se recomienda implementarlas primero, idealmente en 1-2 días.

1. Rate limiting distribuido (Redis)

PRIORIDAD CRÍTICA · ESFUERZO MEDIO

Problema: El rate limiter actual usa `Map()` en memoria. En Vercel, cada serverless function es una instancia separada, así que un atacante tiene 10 intentos por instancia. Con 10 instancias paralelas = 100 intentos por ventana.

Fix: Migrar a Upstash Redis (plan free) o Vercel KV. Cambiar `rateLimitStore = new Map()` por `await redis.incr(key)`. Beneficio: 1 solo contador compartido entre todas las instancias. Costo: \$0 (free tier).

2. Configurar Resend (email service)

PRIORIDAD CRÍTICA · ESFUERZO BAJO

Problema: `RESEND_API_KEY` no está en `.env`. Los emails de verificación y reseteo de contraseña no se envían en producción. Los usuarios no pueden verificar su cuenta ni recuperar contraseña.

Fix: Crear cuenta en resend.com, agregar `RESEND_API_KEY` y `RESEND_FROM_DOMAIN` a Vercel env vars, y verificar el dominio.

3. Rate limit global en middleware

PRIORIDAD CRÍTICA · ESFUERZO MEDIO

Problema: Solo 9 de 73 endpoints tienen `rateLimit()`. Los otros 64 (reservas, caja, configuración, reportes) pueden recibir requests ilimitadas → DoS, fuerza bruta de operaciones, scraping.

Fix: Crear `middleware.ts` que aplique a todas las `/api/*`. Configurar límites por tipo: mutaciones 30 req/min por usuario, queries 100 req/min, endpoints públicos 10 req/min por IP.

4. Cifrar credenciales MP en BD

PRIORIDAD CRÍTICA · ESFUERZO MEDIO

Problema: `PlatformConfig` guarda `mp_access_token` y `mp_webhook_secret` en texto plano. Si alguien accede a la BD (SQL injection, backup robado, insider), tiene las credenciales de MP completas.

Fix: Crear `src/lib/crypto.ts` con `encrypt()/decrypt()` usando AES-256-GCM y una key de `ENCRYPTION_KEY` env var. Cifrar antes de upsert, descifrar al leer. El token quedaría: `enc:abc123...` en la BD.

5. Validar IP en webhook MP

PRIORIDAD CRÍTICA · ESFUERZO BAJO

Problema: El webhook valida la firma HMAC, pero si un atacante descubre el webhook secret (que está en texto plano — ver punto 4), puede forjar webhooks desde cualquier IP.

Fix: MP publica sus rangos de IP. Validar `x-forwarded-for` contra esos rangos antes de procesar el webhook. Capa adicional de defensa si la firma se compromete.

Fase 2 — Mejoras importantes

Estas 7 mejoras aportan defensa en profundidad y visibilidad. Se recomienda implementarlas en 2-3 días, después de la Fase 1.

6. Sentry (monitoreo de errores)

PRIORIDAD IMPORTANTE · ESFUERZO BAJO

Problema: No hay Sentry ni logs centralizados. Si hay un ataque, no te enterás hasta que un usuario reporta el problema.

Fix: `npm i @sentry/nextjs`, configurar `sentry.client.config.ts` y `sentry.server.config.ts`. Captura errores, performance issues, y violaciones CSP. Alertas a email/Slack.

7. Error messages genéricos

PRIORIDAD IMPORTANTE · ESFUERZO BAJO

Problema: Varios endpoints devuelven `error.message` del catch directamente al cliente. Un atacante puede aprender sobre la estructura interna (nombres de tablas, queries).

Fix: Crear helper `handleApiError(error, operation)` que en dev loguea el error completo, en prod devuelve "Error interno" al cliente, y siempre loguea a Sentry con contexto.

8. Quitar stack trace log

PRIORIDAD IMPORTANTE · ESFUERZO TRIVIAL

Problema: `create-checkout/route.ts:67` hace `JSON.stringify(error, Object.getOwnPropertyNames(error), 2)` que loggea el error completo incluyendo propiedades internas que pueden contener tokens.

Fix: Reemplazar por `console.error('[create-checkout] Error:', err.message)` y dejar que Sentry capture el resto.

9. Eliminar auto-migrate.ts

PRIORIDAD IMPORTANTE · ESFUERZO TRIVIAL

Problema: `src/lib/auto-migrate.ts` usa `$executeRawUnsafe` para ejecutar SQL arbitrario (DROP INDEX, ALTER TABLE). No se llama en runtime pero es código peligroso.

Fix: Eliminar el archivo. Las migraciones deben hacerse con `prisma migrate deploy`, no con SQL raw en runtime.

10. Zod para validación de inputs

PRIORIDAD IMPORTANTE · ESFUERZO ALTO

Problema: Los endpoints validan inputs manualmente con `if (!field)` sueltos. Propenso a errores — fácil olvidar validar un campo.

Fix: `npm i zod`, crear schemas declarativos, validar en cada endpoint con `schema.parse(body)`. Type-safe, menos código, validación consistente.

11. CSRF tokens en mutations

PRIORIDAD IMPORTANTE · ESFUERZO MEDIO

Problema: Las API routes usan cookies de sesión. Si hay un XSS, el atacante puede hacer requests con la cookie del usuario. El CSP ayuda pero no es suficiente.

Fix: Generar un token CSRF en el login, incluirlo en headers custom, validar server-side. NextAuth ya protege `/api/auth/*`, pero las demás rutas necesitan protección adicional.

12. Google OAuth config en producción

PRIORIDAD IMPORTANTE · ESFUERZO BAJO

Problema: `GOOGLE_CLIENT_ID` y `GOOGLE_CLIENT_SECRET` no están en `.env`. El login con Google no funcionará en producción.

Fix: Crear OAuth client en Google Cloud Console, agregar env vars a Vercel, configurar `NEXTAUTH_URL` con el dominio de producción real.

Fase 3 — Hardening y resiliencia

Estas 8 mejoras aportan defensa en profundidad, monitoreo y capacidad de recuperación. Se recomienda implementarlas en 3-5 días, pueden correr en paralelo con la Fase 2.

13. Backup automático de BD

HARDENING · ESFUERZO MEDIO

Problema: Neon tiene point-in-time recovery, pero no hay backup exportado a otro proveedor. Si alguien borra la BD de Neon, se pierde todo.

Fix: Script `scripts/backup-db.ts` que haga `pg_dump` y suba a S3/Cloudflare R2 con cifrado. Cron job diario (Vercel Cron o GitHub Actions). Retención de 30 días.

14. Health check endpoint

HARDENING · ESFUERZO BAJO

Problema: No hay un endpoint `/api/health` para integrar con uptime monitoring (UptimeRobot, BetterStack).

Fix: Crear `/api/health` (público) que chequee DB, MP config, y devuelva `{status: 'ok', checks: {db: true, mp: true}}`. Integrar con UptimeRobot (free) para alertas.

15. IP + User Agent en auditoría

HARDENING · ESFUERZO BAJO

Problema: Los registros de Auditoría guardan empleado y detalle, pero no la IP ni el user agent. Si hay un ataque, no se puede rastrear el origen.

Fix: Agregar columnas `ip` y `userAgent` al schema Auditoria. Capturar en cada endpoint que crea auditoría.

16. Dependabot (deps auto-update)

HARDENING · ESFUERZO TRIVIAL

Problema: No hay npm audit automático ni Dependabot. Las vulnerabilidades conocidas pueden quedar sin parchear.

Fix: Activar Dependabot en GitHub: crear `.github/dependabot.yml`. Run `npm audit` en CI. Auto-merge de patches de seguridad.

17. CSP report-uri

HARDENING · ESFUERZO BAJO

Problema: El CSP actual bloquea violaciones pero no las reporta. No se detectan intentos de ataque XSS hasta que uno tiene éxito.

Fix: Agregar `report-uri /api/csp-report` al CSP. Crear endpoint que loguee a Sentry para análisis.

18. Rotación de NEXTAUTH_SECRET

HARDENING · ESFUERZO TRIVIAL

Problema: Si el `NEXTAUTH_SECRET` se filtra, todos los JWTs firmados con él son válidos hasta que expiren (30 días).

Fix: Rotar el secret periódicamente (cada 90 días). Al rotar, todas las sesiones se invalidan automáticamente (los JWTs viejos no validan).

19. 2FA para super-admin

HARDENING · ESFUERZO MEDIO

Problema: El super-admin accede con solo email + contraseña. Si la contraseña se filtra, el atacante tiene control total de la plataforma.

Fix: `npm i otplib`. Al hacer login como super-admin, pedir código TOTP. El super-admin configura su 2FA una vez (QR con Google Authenticator). Obligatorio para super-admin, opcional para usuarios normales.

20. Headers adicionales (COOP, COEP)

HARDENING · ESFUERZO TRIVIAL

Problema: Faltan headers de seguridad recomendados: Cross-Origin-Opener-Policy, Cross-Origin-Embedder-Policy, X-DNS-Prefetch-Control.

Fix: Agregar en `next.config.ts` headers. Defensa en profundidad contra clickjacking avanzado y DNS prefetch leaks.

Orden de implementación recomendado

Fase 1 — Críticas (1-2 días)

- #2 (Resend) — sin esto, register/forgot-password no funcionan en producción
- #9 (eliminar auto-migrate) — 5 minutos, código peligroso
- #8 (quitar stack trace) — 5 minutos
- #1 + #3 (rate limit Redis + global) — protege contra fuerza bruta y DoS

Fase 2 — Importantes (2-3 días)

- #4 (cifrar MP) + #5 (IP webhook) — blindan el sistema de pagos
- #6 (Sentry) + #7 (error genéricos) — visibilidad de ataques
- #12 (Google OAuth) — login con Google en producción

Fase 3 — Hardening (3-5 días)

- #13 (backup) + #14 (health check) — resiliencia
- #16 (Dependabot) + #17 (CSP report) + #20 (headers) — automatización
- #19 (2FA super-admin) — capa final
- #10 (Zod) + #11 (CSRF) + #15 (IP auditoría) — defense in depth

Nota: Todas las mejoras son compatibles con el código existente. Ninguna requiere migración de BD destructiva ni cambio de provider. Se pueden implementar de forma incremental sin downtime.